Goal-Conditioned HyperKAN for Verified Algebraic Rewriting

Complete software specification and implementation checklist

Version 1.0

Prepared for local proof-of-concept on AMD Strix Halo (Linux + ROCm toolbox), with promotion path to single H200/B200 execution.

Document intent	**A build-ready spec for a formally verifiable, visually interpretable neural rewriting system that can be prototyped locally and scaled later.**
Primary claim under test	Whether goal-conditioned spline modulation in a small HyperKAN head improves a verified algebraic rewrite policy relative to matched baselines.
Execution strategy	Phase 1: local Strix Halo proof-of-concept. Phase 2: single H200/B200 scale-up using the same codepath. Phase 3: optional 8x H200 only if the single-GPU result is unusually strong.
Success standard	Held-out exact solve rate, stable search behavior, and repeatable goal-conditioned spline deformation that is visible and correlated with target choice.

Status: disciplined research prototype, not an open-ended moonshot.

Contents

- 1. Executive summary
- 2. Research objective and project boundaries
- 3. Two-phase execution strategy
- 4. Formal task definition
- 5. Symbolic environment and action space
- 6. Dataset specification
- 7. Model architecture
- 8. Training specification
- 9. Inference and verification loop
- 10. Evaluation metrics and success gates
- 11. Visualization requirements
- 12. Local hardware and software environment
- 13. Implementation checklist
- 14. Repository layout
- 15. Promotion path to H200/B200
- 16. Risks, failure modes, and kill criteria
- 17. Stretch path to multi-GPU execution

This document is intentionally opinionated. It preserves the novel part of the project—the goal-conditioned HyperKAN—while stripping away the parts most likely to waste compute without generating evidence.

1. Executive summary

The project builds a formally verifiable neural reasoning system for algebraic rewriting. A current expression S and a target expression G are encoded, a goal-conditioned HyperKAN head predicts the next rewrite action, SymPy executes the action, and exact equivalence testing decides success or failure. The system is constrained to a small action space, explicit search, and deterministic symbolic validation at every step.

The local proof-of-concept is designed for AMD Strix Halo under Linux with an Ubuntu-based ROCm toolbox. The objective is not to maximize model size. The objective is to answer one clean question: does goal-conditioned spline modulation help a verified rewrite policy on symbolic algebra?

The project advances in two practical phases. First, build a small but real local system with exact success metrics, matched baselines, and visual diagnostics. Second, promote the exact same codepath to a single H200 or B200 only if the local signal is strong. An eight-GPU cluster is explicitly out of scope for the initial test.

2. Research objective and project boundaries

2.1 Research objective

Demonstrate that a goal-conditioned HyperKAN action head can produce useful, interpretable, and verifiably correct rewrite policies for symbolic algebra when compared against matched non-KAN baselines.

2.2 What the project is

- A formal rewriting problem with exact symbolic execution and exact end-state verification.
- A controlled comparison between an MLP policy head, a static KAN head, and a goal-conditioned HyperKAN head.
- A visualization-heavy experiment where the learned spline geometry is part of the evidence.

2.3 What the project is not

- A general theorem prover.
- A free-form symbolic language model.
- A claim that KAN universally outperforms standard architectures.
- A justification for using giant compute before the local design has passed a real gate.

3. Two-phase execution strategy

Phase	Hardware	Goal	Output
Phase 1: Local POC	AMD Strix Halo, Linux host, ROCm toolbox	Validate architecture, search loop, exact solve behavior, and visual interpretability.	Working local system, metrics, plots, and solved traces.
Phase 2: Scale-up	Single NVIDIA H200 or B200	Increase dataset size, batch size, and rigor without changing the core design.	Comparable baselines, stronger validation, cleaner figures, deeper trajectories.
Phase 3: Optional expansion	Up to 8x H200 only if justified	Run broader sweeps, larger datasets, or extended ablations after a strong single-GPU result.	Paper-grade sweep capacity, not needed for architectural proof.

The rule is simple: no scale-up unless the local phase proves the architecture is real. Better hardware does not rescue a bad control problem or a sloppy evaluation setup.

4. Formal task definition

4.1 Task statement

Given a current algebraic expression S and a target algebraic expression G, predict the next rewrite action that moves S closer to G. Execute that action with SymPy, update the state, and iterate until exact equivalence with G is reached or the search budget is exhausted.

4.2 Expression families in V1

- Polynomial expressions

- Rational expressions
- Light trigonometric expressions
- Mixed polynomial-rational expressions with limited trigonometric structure

4.3 Explicit exclusions in V1

- Differentiation and integration
- Logarithmic transformations and assumption-heavy identities
- Aggressive power simplification rules
- Large free-form operator sets that make the search graph degenerate

5. Symbolic environment and action space

5.1 Action space for V1

Action	Purpose	Expected failure handling
expand	Expand multiplicative or polynomial structure.	Reject if it is a no-op.
factor	Factor polynomial structure or group terms.	Reject if result is unchanged.
cancel	Cancel common factors in rational expressions.	Reject if not applicable or unchanged.
apart	Perform partial fraction decomposition when valid.	Reject if unsupported or unchanged.
together	Combine fractions under a common denominator.	Reject if unchanged.
trigsimp	Apply trigonometric simplifications under the supported family.	Reject if unchanged or outside the intended family.

5.2 Environment invariants

- Every action wrapper must return either a valid new expression or an explicit invalid/no-op marker.
- Every state must be canonicalized after action execution.
- Every state must receive a canonical hash for duplicate detection and search pruning.
- A successful solve is counted only when SymPy verifies equivalence with the goal.

5.3 Why the action space is intentionally small

The small action space keeps the rewrite graph interpretable, trainable, and visually inspectable. It also limits combinatorial blow-up on local hardware. The goal of V1 is not operator coverage. The goal is clean evidence.

6. Dataset specification

6.1 Generation strategy

Training data is generated by backward construction. Start from a clean goal expression G, apply a controlled sequence of complication transformations to produce a more complex state S, and store intermediate pairs along with all admissible shortest actions back toward G.

6.2 Why backward generation is preferred

- It guarantees known targets.
- It provides valid trajectories without symbolic hallucination.
- It makes exact path length and distance-to-goal available for supervision.
- It avoids relying on arbitrary human-crafted 'simplify this' labels.

6.3 Stored fields per sample

Field	Description
state_str	Canonical structural representation of the current state.
goal_str	Canonical structural representation of the target expression.
valid_shortest_actions	Multi-hot label over the V1 action set.
distance_to_goal	Known shortest remaining distance from the current state to the target.
expr_family	Template family label used for robust splitting and diagnostics.
trajectory_id	Identifier for the source backward-generated trajectory.
canonical_hash	Stable hash for deduplication and search.

6.4 Local dataset target

- Initial bring-up: 20k validated samples
- Primary local training range: 50k to 100k samples
- Trajectory depth: 2 to 6
- Aggressive deduplication and token-length caps

6.5 Split policy

Split by expression family or template, not just random rows. The test set must contain genuinely new combinations rather than near-duplicates from the same trajectory family.

7. Model architecture

7.1 Representation

Use structural tokens derived from SymPy syntax, such as srepr-like or prefix-tree serialization. Avoid human-pretty strings as the primary model input because formatting variation adds noise without adding semantics.

7.2 Shared encoder

- One shared token encoder for both S and G
- Recommended local encoder: token embedding plus 2-layer BiGRU
- Target hidden size: 192 to 256
- Outputs: z_S and z_G

7.3 Feature fusion

Fuse the state and goal representations using concatenation of z_S, z_G, z_S - z_G, and z_S * z_G. This provides direct relative structure without inflating model complexity.

7.4 Baseline A: MLP policy head

The first baseline uses the shared encoder and fused representation, followed by a standard MLP policy head and a small value head. This baseline is mandatory. Without it, there is no credible architectural claim.

7.5 Baseline B: static KAN head

The second baseline replaces the MLP policy head with a small KAN head while removing goal-specific modulation. This tests whether KAN structure alone helps.

7.6 Main model: goal-conditioned HyperKAN

The main model keeps the same encoder and fused features but adds a small hypernetwork driven by z_G. The hypernetwork modulates a compact KAN policy head. The recommendation for V1 is shared spline templates with goal-conditioned mixture weights, because this is cheaper, easier to debug, and much better for visualization than generating every spline coefficient independently.

Component	Recommended V1 design
Encoder	Shared token embedding + 2-layer BiGRU
Fusion	Concatenate z_S, z_G, z_S - z_G, z_S * z_G
Hypernetwork	2-layer MLP on z_G
KAN policy head	2 compact KAN layers with modest spline grid
Goal modulation	Shared spline templates mixed by goal-conditioned weights
Auxiliary head	Distance-to-goal regression head

7.7 Design constraints

- Do not add an unsupervised latent next-state bottleneck in V1.
- Do not let the hypernetwork emit every spline coefficient for the whole network.
- Keep the policy head narrow enough that local iteration remains fast.

8. Training specification

8.1 Objectives

Training is multi-task. The policy predicts admissible shortest next actions, and an auxiliary value head predicts distance-to-goal. This makes search scoring easier and reduces the chance that the policy becomes purely local and myopic.

8.2 Loss

- Action loss: multi-label objective over admissible shortest actions
- Value loss: Huber or smooth L1 on distance-to-goal
- Light entropy regularization to reduce brittle collapse early in training

8.3 Training settings for local POC

- Optimizer: AdamW
- Learning rate: start near 1e-3 and decay on plateau
- Batch size: begin around 128 and adjust to runtime stability
- Use mixed precision only if ROCm stability is acceptable
- Gradient clipping enabled
- Curriculum: start with shorter trajectories before introducing depth 4 to 6

8.4 Required ablations

- MLP baseline
- Static KAN baseline
- HyperKAN main model
- Optional ablation: HyperKAN without value head

9. Inference and verification loop

9.1 Search procedure

Inference is not greedy argmax. Use beam search with explicit state management. A practical local configuration is beam width 4, maximum 8 executed steps, action masking, no-op rejection, and visited-state pruning by canonical hash.

9.2 Beam scoring

- Action log-probability from the policy head
- Penalty or bonus derived from predicted distance-to-goal
- Revisit penalty to discourage loops
- Hard rejection for invalid or no-op actions

9.3 Exact success criterion

A trajectory is successful only when the current symbolic state is verified equivalent to the target by SymPy. One-step action accuracy is useful for debugging. It is not the final metric.

10. Evaluation metrics and success gates

Category	Metric	Why it matters
One-step	Top-1 admissible hit rate	Basic policy quality under the constrained action set.
One-step	Top-3 admissible hit rate	Measures whether search can exploit near-miss policy predictions.
End-to-end	Verified solve rate	The primary metric for the entire project.
End-to-end	Mean solved steps	Shows efficiency, not just correctness.
Search quality	Loop rate and no-op rate	Detects degenerate policy behavior.
Generalization	Solve rate by trajectory depth and expression family	Tests whether performance survives genuinely new structures.

10.1 Local promotion gate

- The local system solves held-out examples end-to-end with exact verification.
- HyperKAN matches or beats the MLP baseline on at least one important end-to-end metric.
- Goal swaps produce repeatable changes in both action distributions and spline geometry.
- Results survive a family-based split rather than collapsing on a random-row split.

11. Visualization requirements

11.1 Required figure set

- Training and validation loss curves
- Solve-rate-by-depth comparison across MLP, static KAN, and HyperKAN
- Goal-swap action heatmaps for fixed states with varying goals
- Spline morph panels showing the same KAN edges under multiple goals
- Verified trajectory graphs for successful and failed search traces

11.2 Why the visualizations matter

The project is not only about final solve rate. It is also about whether goal-conditioned spline modulation is observable and functionally tied to the target. The morphing spline panel is therefore a core evidence artifact, not a decorative extra.

12. Local hardware and software environment

12.1 Hardware target

Primary local environment: AMD Strix Halo with 128 GB shared memory, Linux host, and ROCm user-space inside a toolbox or container.

12.2 Recommended system arrangement

- Arch Linux host retained for the workstation environment
- Ubuntu-based toolbox or container for the ROCm and Python stack
- Host kernel kept in the supported range for the Ryzen AI Max / Strix Halo support path
- Pinned Python environment exported for reproducibility

12.3 Mandatory smoke tests before model work

- PyTorch can allocate tensors on the GPU path
- Forward and backward passes succeed on a tiny model
- SymPy action wrappers behave deterministically
- Multiprocess data generation does not deadlock
- A short train loop can run for at least 100 iterations without instability

13. Implementation checklist

13.1 Phase 0: environment and scaffolding

1. Freeze the host kernel and toolbox/container versions.
2. Create the repository layout and config files.
3. Run ROCm and PyTorch smoke tests.

4. Set up logging, checkpoints, and artifact directories.

13.2 Phase 1: symbolic environment

5. Implement canonicalization, hashing, and equivalence checking.
6. Wrap the six V1 actions with no-op rejection and applicability checks.
7. Unit-test action wrappers on manually curated expressions.
8. Validate that search-state deduplication works.

13.3 Phase 1: dataset generation

9. Build the backward trajectory generator.
10. Generate a 20k-sample bring-up dataset.
11. Measure duplicate rate, token lengths, and depth distribution.
12. Expand to 50k to 100k samples once validation is clean.

13.4 Phase 1: tokenization and dataloading

13. Implement structural tokenization.
14. Build the vocabulary and sequence packing rules.
15. Create deterministic train, validation, and test splits by family.
16. Write batch collation and padding logic.

13.5 Phase 1: baseline training

17. Train the shared-encoder MLP baseline.
18. Run one-step and end-to-end evaluation.
19. Save solved and failed trajectory traces for inspection.

13.6 Phase 1: KAN experiments

20. Train the static KAN baseline with matched encoder and data.
21. Train the goal-conditioned HyperKAN model.
22. Compare runtime, stability, and solve behavior against the MLP baseline.

13.7 Phase 1: visualization and gate decision

23. Generate the required plot set.
24. Inspect solved trajectories manually.
25. Decide whether the HyperKAN effect is real enough to justify scale-up.

13.8 Phase 2: single H200/B200 promotion

26. Freeze the local design and keep the codepath unchanged.
27. Scale dataset size, batch size, and evaluation rigor.
28. Repeat baselines and ablations with stronger bookkeeping.
29. Produce final figures, tables, and curated examples.

14. Repository layout

Path	Purpose
configs/	Experiment and hardware configuration files.
data_gen/	Action wrappers, canonicalization, dataset generation, validation.

tokenizer/	Structural tokenization and vocabulary logic.
models/	Shared encoders, MLP head, KAN head, HyperKAN head.
train/	Losses, training loops, checkpoint management.
search/	Beam search, masking, and trajectory management.
eval/	One-step evaluation and verified end-to-end evaluation.
viz/	Plots for splines, heatmaps, and trajectory graphs.
scripts/	Local and scale-up launch scripts and environment helpers.
artifacts/	Checkpoints, plots, logs, and solved examples.

15. Promotion path to H200/B200

15.1 What changes

- More data
- Larger batch sizes
- Slightly wider encoders or longer sequence caps if justified
- More seeds and more careful ablations

15.2 What must not change

- Action space for the main comparison
- Dataset schema
- Structural tokenization
- Evaluation protocol
- Core baseline set
- Exact symbolic verification logic

15.3 Why a single H200/B200 is enough

For this project, a single datacenter GPU is enough to test whether the local signal scales. The model class and dataset sizes are modest by modern standards. Multi-GPU scale only becomes justified once the single-GPU run proves the architecture is worth a broader sweep.

16. Risks, failure modes, and kill criteria

Risk	Failure mode	Response
Degenerate search graph	The model oscillates between inverse-like actions or accumulates no-ops.	Keep the action set small, mask invalid actions, and use visited-state penalties.
Dataset leakage	Random-row splits inflate performance.	Split by family or template and inspect held-out structures.
HyperKAN is only cosmetic	Spline plots look interesting but end-to-end metrics do not improve.	Treat as a negative result and report it honestly.
ROCm instability	Mixed precision or large batch sizes	Fall back to conservative precision

	become unreliable locally.	and reduce batch size.
Baseline collapse	Even the MLP baseline cannot solve nontrivial examples.	Revisit the environment, labels, and search setup before blaming the architecture.

16.1 Kill criteria

- The MLP baseline cannot solve nontrivial held-out examples after a fair setup.
- The search graph is dominated by loops or no-ops despite masking and canonicalization.
- The HyperKAN produces prettier plots but no measurable policy benefit.
- Performance collapses when evaluation uses family-based splits.

17. Stretch path to multi-GPU execution

Eight H200s are not part of the initial proof. They become interesting only if the single-GPU run suggests one of the following: materially larger action spaces, larger search budgets, much larger datasets, or an ablation suite broad enough to justify the additional infrastructure.

The default stance is conservative: earn the right to scale.

Appendix A. Phase summary at a glance

Stage	Minimum evidence required before advancing
Local environment ready	Stable GPU path, deterministic symbolic environment, dataset generator validated.
Local architecture validated	Nontrivial verified solves, matched baselines run, interpretable goal-conditioned spline changes.
Single-GPU scale-up justified	Local promotion gate cleared with reproducible metrics and useful plots.
Multi-GPU justified	Single H200/B200 result is strong enough that larger sweeps or broader task scope are genuinely warranted.